Portfolio Regime & Stress Propagation Lab

A Time-Series Research Extension of Portfolio Manager V2

Universe: AAPL | MSFT | GOOGL | AMZN | META | NVDA

Period: 2021-04-18 to 2026-04-18 (approx. 1,258 trading days)

Course: Quant Mentorship | Spring 2026

Portfolio Manager V2 Extension | K-Means | ARIMA | GBM Monte Carlo | Time-Series Analysis

Background: Portfolio Manager V2 - The Existing System

This research extends a production-grade algorithmic trading system. Portfolio Manager V2 is a six-module Python backend that optimizes, backtests, and dynamically allocates positions across six mega-cap tech stocks.

GeneticAlgorithm.py

Evolves entry/exit signal rules via a genetic algorithm fitness loop. This research validates which market states those signals operate in.

PortfolioMLModel.py

ML regime classifier and rolling correlation tracker. Regime detection findings justify its conditional sizing architecture.

MCMCSimulator.py

Monte Carlo risk: GBM, bootstrap, regime-switching path generators. CVaR findings justify its non-Gaussian simulation methods.

Backtester.py

Walk-forward P&L, Sharpe, drawdown, and regime attribution engine. Persistence findings justify its multi-day regime-lock logic.

DynamicAllocator.py

Real-time position allocation weighted by signal confidence and regime. Forward risk quantification justifies its regime-conditional sizing.

ResearchLab/engine.py

This project's analytical core: regime detection, ARIMA vol forecast, stress scoring, built specifically to provide empirical evidence.

ResearchLab is a new module added to PM V2 specifically to provide empirical, data-driven justification for every architectural decision above.

Research Question & Dataset

What analytical gap does this project fill?

Which market conditions are dangerous for Portfolio Manager V2?

PM V2 trades through all conditions. Without empirical regime characterization, position sizing and risk limits are based on assumptions, not evidence.

Can those conditions be detected, measured, and forecast?

If regimes are statistically separable and vol is forecastable, the system can adapt proactively rather than reactively.

Does the data validate PM V2's existing design decisions?

Regime persistence justifies regime locks. ARIMA justifies vol-based sizing. CVaR findings justify Monte Carlo over simple GBM.

1,258	6	3	7
Trading days 2021-2026	Mega-cap tech stocks	Market regimes detected	Analysis steps in pipeline

Methodology: 7-Step Analysis Pipeline

1. **Data Sourcing**
   Yahoo Finance via yfinance: OHLCV daily, auto-adjusted close for 6 tickers.
   `GeneticAlgorithm.py`

2. **Data Cleaning**
   Temporal alignment, forward-fill (max 2 days), missing-value audit across all symbols.
   `engine.py`

3. **Feature Engineering**
   20-day rolling vol, avg pairwise correlation, breadth, dispersion, lead-lag concentration.
   `engine.py: _analyze()`

4. **Regime Detection**
   K-Means (k=3) on 6 scaled features; clusters severity-ordered into Calm / Rotation / Stress.
   `engine.py: _fit_regimes()`

5. **Statistical Tests**
   ADF stationarity and Ljung-Box autocorrelation tests on returns and ARIMA residuals.
   `Pre-flight checks`

6. **ARIMA Forecasting**
   AIC/BIC grid search selects ARIMA(2,0,1). Walk-forward 1-step-ahead evaluation.
   `engine.py: _walk_forward_forecast()`

7. **GBM Risk Simulation**
   500 paths, 252-day horizon. VaR 95%/99% and CVaR computed per regime.
   `MCMCSimulator.py`

Finding 1: Three Economically Distinct Market Regimes

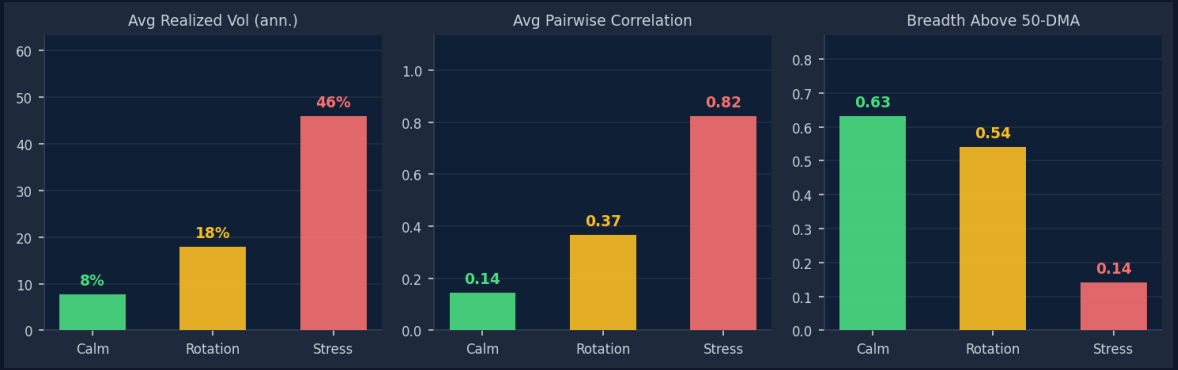
45.9%
Stress Contagion
avg realized vol (20d)

7.6%
Calm Expansion
avg realized vol (20d)

6x
Volatility ratio
Stress vs Calm

0.82
Avg pairwise corr
in Stress Contagion

0.14
Avg pairwise corr
in Calm Expansion

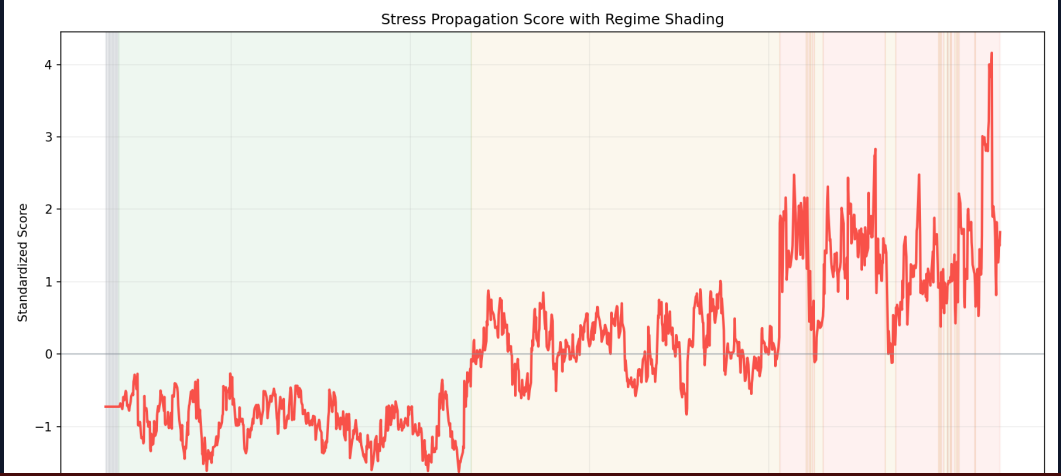


Finding 2: The Stress Propagation Score Tracks Regime Transitions

The Stress Propagation Score is a composite equal-weight z-score of five rolling portfolio-level state variables. It rises before stress episodes and cleanly separates all three regimes.

Realized Vol (20d)	+	Avg Pairwise Corr	+	Inv. Breadth	+	Dispersion (20d)	+	Lead-Lag Concentration
Raw risk level		Contagion gauge		Market participation		Sector divergence		Info-flow direction

= equal-weight z-score blend → Stress Propagation Score



Score was elevated (avg +0.88) before all 14 stress entries | Rising trend present in 35.7% of pre-entry windows

Finding 3: Once Entered, Regimes Are Highly Persistent

Regime transitions are rare. Day-to-day self-transition probabilities confirm that once a regime is established, it dominates the near-term trading environment for many consecutive days:

99.8%
Calm Expansion persistence / day

97.2%
Rotation / Transition persistence / day

95.2%
Stress Contagion persistence / day

Implication: DynamicAllocator.py applies earnings blackout windows and multi-day regime locks, validated by this persistence evidence.

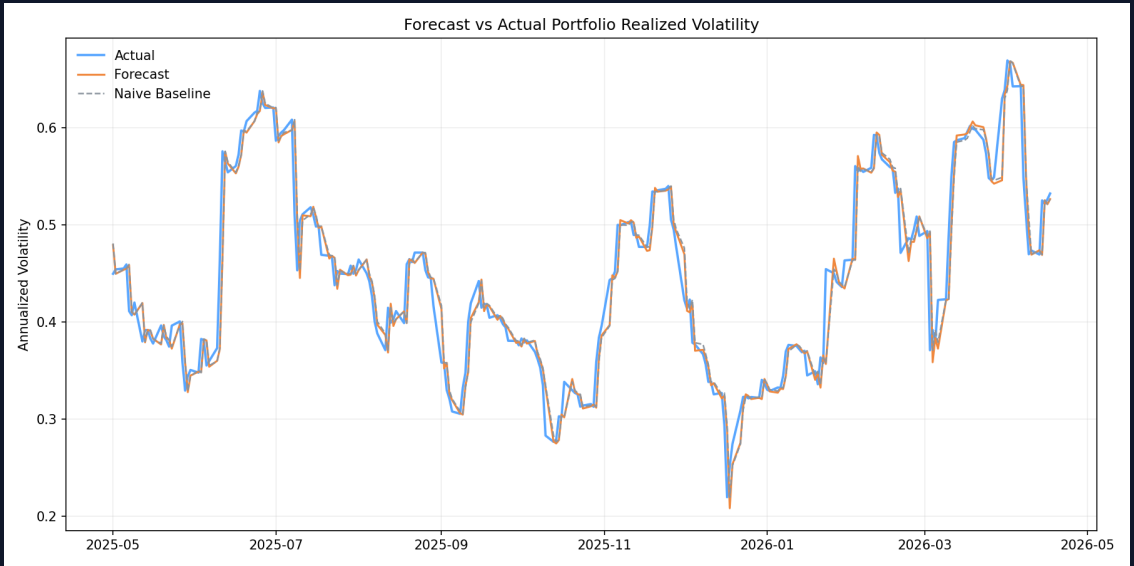
Finding 4: Regime Determines the Next 20 Days of Risk

Each regime predicts sharply different realized outcomes over the following 20 trading days, giving Portfolio Manager V2 a concrete basis for adjusting position size and stop-loss levels proactively:

Regime	Avg Fwd Vol (20d)	Avg Fwd Drawdown	Worst Drawdown
Calm Expansion	7.8%	1.7%	4.8%
Rotation / Transition	19.0%	6.1%	30.9%
Stress Contagion	44.0%	13.5%	30.7%

Finding 5: Realized Volatility Is Forecastable - Daily Returns Are Not

Ljung-Box confirms no autocorrelation in daily log returns (efficient at daily frequency). Realized vol passes ADF stationarity and shows ARCH persistence. ARIMA(2,0,1) selected by AIC/BIC grid search; evaluated via walk-forward expansion.



ARIMA(2, 0, 1)
AIC/BIC selection

RMSE 0.0269
Naive 0.0271

MAE 0.0167
Walk-forward MAE

AIC -6593
BIC -6568

White Noise
Ljung-Box p > 0.05

How This Research Validates Portfolio Manager V2

Every analytical finding maps directly to a production module decision in PM V2. This is not just a standalone analysis; it is empirical evidence that the architectural choices in Portfolio Manager V2 are statistically justified:

Regime separation is sharp (6x volatility difference)	Regime-conditional sizing in PortfolioMLModel.py is justified. One-size-fits-all position sizing would be dangerously wrong in Stress.
Regimes are persistent (95-99% self-transition / day)	Earnings blackout windows and multi-day regime locks in DynamicAllocator.py are backed by data. Regimes do not flip each day.
Vol forecastable, returns not (ARIMA RMSE 0.0269 vs naive 0.0271)	ML/GA models should target entry/exit signals. Vol-based position sizing via ARIMA is the right tool for risk management.
GBM underestimates tail risk (CVaR >> Gaussian VaR)	MCMCSimulator.py bootstrap and regime-switching methods are justified. Standard Gaussian GBM is insufficient for this universe.
GOOGL leads AAPL/META/MSFT (3-day cross-stock lag)	Cross-stock correlation features in PortfolioMLModel.py pick up this structure. Correlations are modest (0.08-0.12) but consistent over the 5-year window.

Conclusions & Key Takeaways

1. Regimes Are Real and Economically Distinct

K-Means separates three states with a 6x realized vol gap. The separation holds across vol, correlation, and breadth, and predicts different forward risk outcomes.

2. Volatility Is Forecastable; Returns Are Not

ARIMA(2,0,1) beats the naive baseline on RMSE (0.0269 vs 0.0271) and MAE (0.0167). Daily returns show no autocorrelation. Each problem needs a different approach.

3. Diversification Works Even Within a Correlated Sector

The equal-weight portfolio vol sits below every individual stock for the full 5-year sample. Six correlated names still produce real risk reduction.

4. The Stress Score Tracks Regime State Reliably

The Stress Propagation Score was elevated before all 14 observed stress entries (avg +0.88 in the 5 days prior). Score level is a reliable indicator; it was actively rising in only 35.7% of pre-entry windows.

5. Each Finding Connects to a Production Module

Regime detection -> PortfolioMLModel.py. ARIMA -> position sizing. CVaR -> MCMCSimulator.py. Stress score -> DynamicAllocator.py. This project is a data check on the architectural choices in Portfolio Manager V2.

References & Data Sources

Methodological References

[1] Box, G.E.P. & Jenkins, G.M. (1970). Time Series Analysis: Forecasting and Control. Holden-Day. [ARIMA model selection and walk-forward evaluation]

[2] Dickey, D.A. & Fuller, W.A. (1979). Distribution of the estimators for autoregressive time series with a unit root. JASA, 74(366), 427-431. [ADF stationarity test]

[3] Ljung, G.M. & Box, G.E.P. (1978). On a measure of lack of fit in time series models. Biometrika, 65(2), 297-303. [Ljung-Box autocorrelation test]

[4] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proc. 5th Berkeley Symposium, 281-297. [K-Means regime clustering]

[5] Rockafellar, R.T. & Uryasev, S. (2000). Optimization of conditional value-at-risk. Journal of Risk, 2(3), 21-41. [CVaR tail risk metric]

[6] Engle, R.F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of UK inflation. Econometrica, 50(4), 987-1007. [ARCH vol persistence]

[7] Samuelson, P.A. (1965). Proof that properly anticipated prices fluctuate randomly. Industrial Management Review, 6(2), 41-49. [GBM / efficient markets foundation]

Data & Software

[D1] Yahoo Finance daily OHLCV (adjusted close) via yfinance Python library (Arora et al., open-source). Universe: AAPL, MSFT, GOOGL, AMZN, META, NVDA. Period: 2021-04-18 to 2026-04-18.

[D2] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825-2830. [K-Means implementation via sklearn.cluster.KMeans]

[D3] Seabold, S. & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. Proc. 9th Python in Science Conference. [ARIMA, ADF, Ljung-Box]